

ASYMPTOTIC NOTATION

Prof. Madonna Lamin, ITMBU

ALGORITHM EFFICIENCY USING ASYMPTOTIC NOTATION

Big O, Big Ω , and Big Θ – From Fundamentals to Real Computer Science Applications

The goal is not merely to memorize definitions, but to develop the ability to **look at an algorithm, count its work, express its efficiency mathematically, and compare it with other algorithms.**

WHY DO WE ANALYZE ALGORITHMS

Suppose two programmers write programs to search for a student ID.

Algorithm A

Checks every student one by one.

Algorithm B

Uses a sorted list and repeatedly divides the search space into half.

CONTD...

For 10 students, both may appear fast.

For **10 million students**, the difference becomes enormous.

So the important question is:

How does the running time of an algorithm grow as the input size grows?

This is the central idea behind **algorithm analysis**.

WHAT IS INPUT SIZE?

We normally represent the size of input by **n**.

Problem	Input Size n
Searching in an array	Number of elements
Sorting students/contacts	Number of students/contacts
Processing a string	Number of characters
Graph Traversal	Number of vertices/edges
Matrix Multiplication	Matrix dimension
Processing database records	Number of records

CONTD...

For example:

Student IDs:

101 205 309 411 523 602 715

There are 7 elements.

Therefore:

$N = 7$

If there are 1,000,000 students:

$N = 1,000,000$

WHAT DOES EFFICIENCY MEAN?

Algorithm efficiency generally concerns:

1. **Time efficiency** — how much computation is required?
2. **Space efficiency** — how much memory is required?

In this session we concentrate on **time complexity**.

CONTD...

Consider:

Algorithm A $\rightarrow 10n$ operations

Algorithm B $\rightarrow n^2$ operations

When $n=10$:

- $10n = 100$
- $n^2 = 100$

They are approximately equal

But when $n = 1,000,000$

- $10n = 10,000,000$
- While $n^2 = 1,000,000,000,000$

The difference is enormous.

That is why growth matters

THE BASIC IDEA BEHIND ASYMPTOTIC ANALYSIS

Asymptotic analysis studies the behavior of an algorithm when:

$n \rightarrow \infty$

In simple words,

What happens to the algorithm when the input becomes very large?

We generally ignore:

- machine speed
- programming language
- compiler
- small constant differences
- lower-order terms

and focus on the **dominant growth term**.

EXAMPLE- SIMPLIFYING A RUNNING TIME FUNCTION

Suppose an algorithm takes:

$$T(n) = 3n^2 + 5n + 10$$

For very large n , which term dominates?

$$3n^2$$

Therefore,

$$T(n) = \Theta(n^2)$$

We usually ignore:

- coefficient 3
- $5n$
- constant 10

because the quadratic term dominates as n grows

COMMON GROWTH RATES

Complexity	Name	Example
$O(1)$	Constant	Array Access
$O(\log n)$	Logarithmic	Binary Search
$O(n)$	Linear	Linear Search
$O(n \log n)$	Linearithmetic	Merge Sort
$O(n^2)$	Quadratic	Bubble Sort
$O(n^3)$	Cubic	Basic matrix multiplication
$O(2^n)$	Exponential	Some recursive subset algorithms
$O(n!)$	Factorial	Brute-force travelling salesperson problem

A USEFUL GROWTH HIERARCHY

$$1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!$$

As n becomes large, algorithms further to the right generally become much more expensive.

THREE IMPORTANT ASYMPTOTIC NOTATIONS

The three fundamental notations are:

1. Big O : $O(g(n))$
2. Big Omega : $\Omega(g(n))$
3. Big Theta : $\Theta(g(n))$

They describe different types of bounds.

BIG-O UPPER BOUND

Fundamental Idea:

Big O tells us:

The algorithm will not grow faster than a particular rate, asymptotically.

Formally: $f(n) = O(g(n))$

if there exist positive constants c and n_0 , such that

$$0 \leq f(n) \leq cg(n)$$

for all:

$$n \geq n_0$$

CONTD..

Interpretation: "At most this order of growth."

For example:

$$T(n) = 3n + 5$$

We can say: $T(n) = O(n)$

because eventually $3n+5$ is bounded above by some constant multiple of n .

REAL APPLICATION

Suppose we have,

Student IDs:

101 205 309 411 523 602 715

We want to find:

523

Linear search checks:

101 → 205 → 309 → 411 → 523

If there are n students, in the **worst case** we may **check all n** .

Therefore, $T(n)=n$ and $O(n)$

CONTD..

Real application

Linear search can be used when:

- data is unsorted
- dataset is small
- simplicity is more important than speed

BIG Ω NOTATION- LOWER BOUND

Big Omega tells us:

The algorithm requires at least this much growth asymptotically.

Formally, $f(n) = \Omega(g(n))$

if there exist positive constants c and n_0 , such that
 $0 \leq c g(n) \leq f(n)$ for all $n \geq n_0$

CONTD..

Interpretation: "At least this much work."

For example:

$$f(n) = 3n + 5$$

Then $f(n) = \Omega(n)$

because the function grows at least linearly.

REAL APPLICATION: FINDING THE MINIMUM

Consider:

10 45 23 78 12

To determine the maximum, we must examine every element.
Why?

Because suppose we have examined:

10 45 23

We cannot know whether 78 or another unseen element is larger.

Therefore, any correct algorithm must inspect all n elements.

Hence, **$T(n) = \Omega(n)$**

This is an important example because the lower bound is fundamental to the problem.

BIG Θ - TIGHT BOUND

Big Theta tells us:

The algorithm grows exactly at this asymptotic rate, within constant factors.

Formally, $f(n) = \Theta(g(n))$ if there exist positive constants c_1, c_2, n_0 such that $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$

CONTD...

"This is the actual asymptotic growth rate."

For example:

$$f(n) = 5n + 10$$

Then, $f(n) = \theta(n)$

because it both $O(n)$ and $\Omega(n)$

Therefore, $\theta(n)$

THE RELATIONSHIP BETWEEN O , Ω AND Θ

Think of it like
inequalities between
numbers

Notation	Analogy	Meaning
$f(n) = O(g(n))$	$f \leq g$	Upper bound
$f(n) = \Omega(g(n))$	$f \geq g$	Lower bound
$f(n) = \Theta(g(n))$	$f = g$	Tight bound (both hold)

So **Θ is the intersection of O and Ω** — it's the "sweet spot" where the same function bounds growth from both above and below (within constant factors).

CONTD...

Suppose an algorithm's runtime is $f(n) = 3n^2 + 5n + 2$.

- $O(n^2)$ – true, since $f(n)$ grows no faster than n^2
- $O(n^3)$ – also true (a looser but still valid upper bound)
- $\Omega(n^2)$ – true, since $f(n)$ grows no slower than n^2
- $\Omega(n)$ – also true (a looser but still valid lower bound)
- $\Theta(n^2)$ – true and most informative, since n^2 is both a tight upper *and* lower bound

Notice that O and Ω bounds aren't unique – many valid ones exist, some tighter than others. Θ , when it applies, pins the growth rate down precisely.

CONTD..

Why this matters in practice?

O is used most often in casual conversation ("this algorithm is $O(n \log n)$ ") because people usually care about worst-case performance guarantees.

Ω is used when arguing that a problem *cannot* be solved faster than some rate (e.g., "comparison-based sorting is $\Omega(n \log n)$ ").

Θ is the most precise statement and is preferred in formal analysis when the upper and lower bounds coincide – it says "this is *exactly* the growth rate, not just a ceiling or floor."

WHY DO WE IGNORE CONSTANTS?

Suppose: $T1(n)=5n$ and $T2(n)=100n$

Technically, $T2$ requires more operations.

But both grow linearly.

Therefore, $T1(n)=\theta(n)$ and $T2(n)=\theta(n)$

Asymptotic notation focuses on **growth rate**, not exact execution time.

WHY DO WE IGNORE LOWER ORDER TERMS?

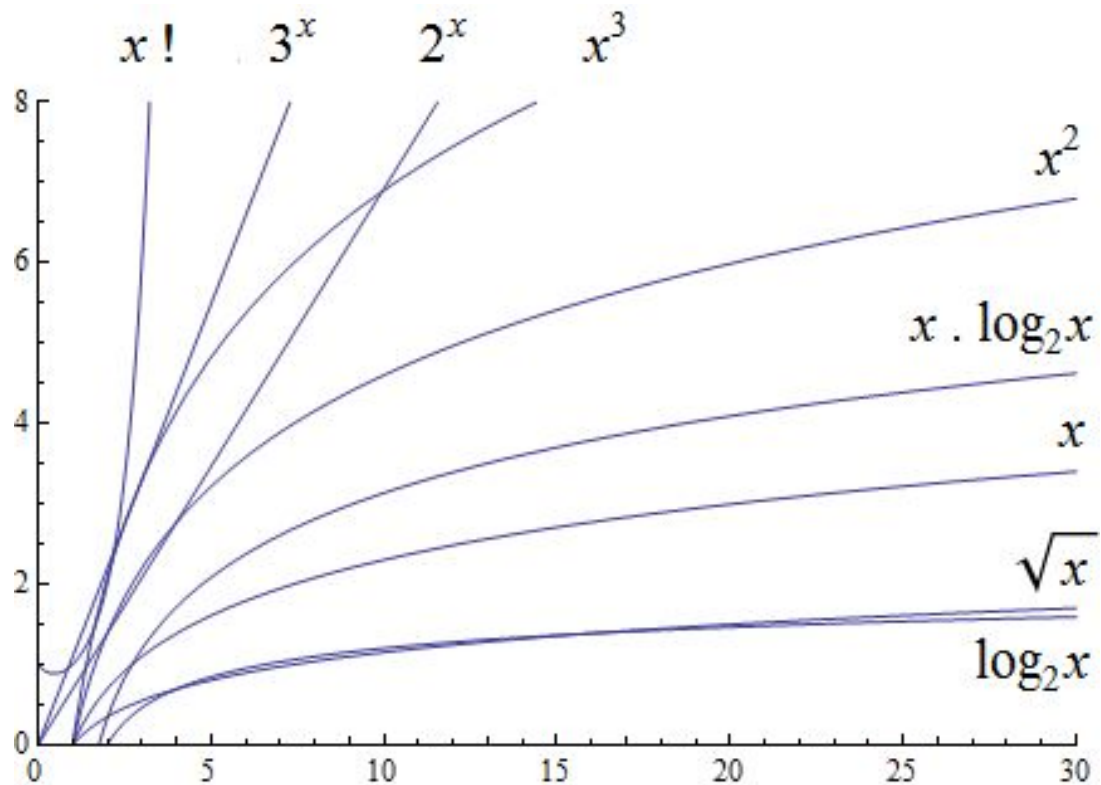
Consider:

$$T(n) = n^3 + 100n^2 + 500n + 1000$$

For very large n : n^3 dominates the other terms, therefore

$$T(n) = \theta(n^3)$$

GRAPHICAL REPRESENTATION



Refer:

https://ggc-discrete-math.github.io/growth_functions.html

CONTD...

As n grows:

- $O(1)$ barely grows
- $O(\log n)$ grows slowly
- $O(n)$ grows steadily
- $O(n \log n)$ grows faster
- $O(n^2)$ grows rapidly

This is why choosing the right algorithm matters.

APPLICATION: ARRAY ACCESS

Consider: `x = arr[500];`

If we know the index, the computer can directly calculate the memory address.

Therefore, $T(n) = \theta(1)$

This is constant time

Whether the array contains 19 elements or 10 million elements, accessing one known index takes essentially constant-time work

APPLICATION 2: LINEAR SEARCH

```
for(i = 0; i < n; i++)  
{  
    if(arr[i] == key)  
        return i;  
}
```

In the worst case, n , elements are examined.

Therefore, $O(n)$

But there is an important distinction:

Best Case: The key is at the first position : $\Theta(1)$

Worst Case: The is at the last position or absent: $O(n)$

Average Case: Usually, $\Theta(n)$, under common assumptions about the key's location

APPLICATION 3: BINARY SEARCH

Suppose the sorted data contains:

10 20 30 40 50 60 70 80

We search for : 70

Binary search repeatedly halves the search space

8 elements



4 elements



2 elements



1 element

CONTD..

After k steps: $2^k/n = 1$

Therefore, $n = 2^k$

Taking logarithm: $k = \log n$

Hence, $T(n) = \theta(\log n)$

WHY IS BINARY SEARCH SO POWERFUL?

Suppose we have,

$n=1,000,000$

Linear Search may require approximately: 1,000,000

Binary Search requires approximately: $\log_2(1,000,000) \approx 20$
comparisons

That is a huge difference

REAL APPLICATIONS OF BINARY SEARCH

Binary-Search-like techniques are fundamental in:

- Database indexes
- Searching sorted datasets
- Dictionaries
- Library lookup systems
- Range Searching
- Debugging/Version Search problems

APPLICATION 4: NESTED LOOP

```
for(i = 0; i < n; i++){  
    for(j = 0; j < n; j++){  
        printf("*");  
    }  
}
```

Outer loop: n iterations

Inner loop: n iterations

Therefore, $n \times n = n^2$

Hence, $\Theta(n^2)$

REAL APPLICATION: COMPARING EVERY PAIR

Suppose we have students: A, B, C, D

Possible comparisons: A-B, A-C, A-D, B-C, B-D, C-D

The number of pairs is approximately: $n(n-1)/2$

Therefore: $\Theta(n^2)$

CONTD...

Applications:

This occurs in:

- duplicate detection
- brute-force similarity comparison
- pairwise distance calculations
- collision checking
- some graph algorithms

APPLICATION 5: THREE NESTED LOOPS

```
for(i = 0; i < n; i++)  
    for(j = 0; j < n; j++)  
        for(k = 0; k < n; k++)  
            operation();
```

Number of operations: $n \times n \times n$

Therefore, $\Theta(n^3)$

APPLICATION: MATRIX MULTIPLICATION

For two $n \times n$ matrices, the basic matrix multiplication algorithms performs approximately: n^3

Therefore, $\theta(n^3)$

This illustrates why algorithmic improvements in matrix computation are so important in:

- computer graphics
- scientific computing
- machine learning
- simulations
- data science

APPLICATION 6: MERGE SORT

Merge Sort follows:

Divide



Sort left half



Sort right half



Merge

CONTD...

The array is repeatedly divided: $n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots$

Number of levels: $\log n$

At each level, approximately n work is performed

Therefore, $n \times \log n$

and $\theta(n \log n)$

CONTD..

Many computations are repeated.

The running time is exponential, commonly expressed as :
 $O(2^n)$

Recursion can create enormous repeated work.

THE SAME PROBLEM CAN HAVE DIFFERENT COMPLEXITY

Consider Fibonacci:

Naive Recursive Fibonacci: $O(2^n)$

Iterative Fibonacci: $O(n)$

Dynamic Programming: $O(n)$

With $O(n)$ additional space in the basic memoized/table approach.

This is one of the most important lessons in algorithm design

The problem does not determine the complexity; the algorithm does.

IMPORTANT PROPERTIES OF ASYMPTOTIC NOTATION

Property 1: Constant Multiplication

If $f(n) = \theta(g(n))$ then,

for any positive constant c :

$$c(f(n)) = \theta(g(n))$$

Example:

$$5n = \theta(n)$$

$$100n = \theta(n)$$

CONTD..

Property 2: Ignore lower-order terms

If $f(n) = n^2 + n + 1$, then:

$$f(n) = \Theta(n^2)$$

because n^2 dominates n and 1

CONTD...

Property 3: Transitivity

If $f(n) = O(g(n))$ and $g(n) = O(h(n))$ then:

$$f(n) = O(h(n))$$

Example:

$$n = O(n \log n) \text{ and } n \log n = O(n^2)$$

$$\text{Therefore, } n = O(n^2)$$

CONTD...

Property 4: Reflexivity

Every function is asymptotically bounded by itself

Therefore, $f(n) = O(f(n))$

Also, $f(n) = \Omega(f(n))$

and , $f(n) = \Theta(f(n))$

CONTD...

Property 5: Symmetry of Θ

If $f(n) = \Theta(g(n))$ then:

$$g(n) = \Theta(f(n))$$

Example:

$$3n + 5 = \Theta(n)$$

Therefore,

$$n = \Theta(3n + 5)$$

CONTD...

Property 6: Sum Rule

Suppose, $f(n) = O(g(n))$ and $h(n) = O(k(n))$ then

$$f(n) + h(n) = O(g(n) + k(n))$$

In practical algorithm analysis, we often use:

$O(n) + O(n^2)$ and retain the dominant term $O(n^2)$

CONTD...

Property 7: Product Rule

If two operations occur inside nested loops, their complexities multiply.

```
for(i=0; i<n; i++){  
    for(j=0; j<n; j++)    {  
        operation();  
    }  
}
```

Outer: $O(n)$

Inner: $O(n)$

Therefore, $O(n) \times O(n) = O(n^2)$

SEQUENTIAL VERSUS NESTED OPERATIONS

Sequential

A

B

Complexity: $T(n) = T_A(n) + T_B(n)$

Nested

A

B

Complexity: $T(n) = T_A(n) \times T_B(n)$

CONTD..

Property 8: Logarithm Base Does Not Matter Asymptotically

Consider, $\log_2 n$ and $\log_{10} n$

Using change of base:

$$\log_2 n = \log_{10} n / \log_{10} 2$$

Since: $1/\log_{10} 2$ is a constant

$$\log_2 n = \log_{10} n$$

Therefore, in asymptotic notation we usually write: $O(\log n)$ without worrying about the base.

CONTD...

Property 9: Polynomial Growth

For fixed positive constants : n^a grows faster than n^b if,
 $a > b$

For example:

$$n < n^2 < n^3$$

Therefore: $O(n) < O(n^2) < O(n^3)$ in terms of growth rate